

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

Department of Computer Science & Engineering

LABORATORY ASSIGNMENT REPORT ASSIGNMENT 6: TAGS AND TEST EXECUTION

Name	:	Ashmita Saha
Enrollment no.	:	12023002028118
Department	:	CSE (AI & ML)
Course	:	Python for Automation
Academic Session	:	2023-2027
Submission Date	:	28 Sept, 2026

1. MODULE NAME & EXPERIMENT TITLE

Module Name: Tags and Test Execution

Experiment Title: Tag-Based Test Selection using `--include` and `--exclude` Options and Parallel Test Execution using `--processes` / `pabot` in Robot Framework

2. PROBLEM STATEMENT

Implement tagged test suites in Robot Framework and execute targeted test runs based on specific criteria:

- Assign tags to individual test cases and test suites (e.g., smoke, sanity, regression, slow).
- Execute only selected subsets of tests using the CLI options `--include` and `--exclude`.
- Run tests concurrently across multiple parallel worker processes using the `--processes` option (via `pabot` runner).

3. OBJECTIVE

- To categorize and organize test suites using Robot Framework tags for modular test management.
- To perform conditional test execution by filtering tests with `--include` and `--exclude` command-line flags.
- To reduce overall execution duration by executing independent test suites in parallel utilizing multiple system processes.

4. TOOLS, SOFTWARE, AND CONCEPTS USED

- **Programming Language & Engine:** Python 3.11.x, Robot Framework 6.1
- **Parallel Execution Tool:** `pabot` (Parallel executor for Robot Framework)
- **Development Environment:** Visual Studio Code (VS Code Terminal & Editor)
- **Key Concepts:** Metadata Tagging (`[Tags]`), Command Line Filtering (`--include / -i`, `--exclude / -e`), Process Parallelization (`--processes / -p`).

5. SOURCE CODE / IMPLEMENTATION STEPS

Tagged Test Suite Script (`assignment6_tags.robot`)

```
assignment6_tags.robot - Tagged Test Suite                                Robot Framework

1  *** Settings ***

2  Documentation Assignment 6 - Test Tagging and Parallel Execution Demo

3  Default Tags regression

4

5  *** Test Cases ***

6  TC_01: User Authentication Check
```

6. OUTPUT WITH CLEARLY LABELLED SCREENSHOTS

The code was executed in VS Code terminal under three execution scenarios: tag inclusion, tag exclusion, and multi-process parallel execution.

Screenshot 1: Tag Filtering Execution via --include and --exclude

```
PS C:\Users\Ashmita\Automation_Lab> robot --include sanity assignment6_tags.robot
===== Assignment6 Tags
===== Executing TC_01:
High-priority authentication test TC_01: User Authentication Check | PASS | Executing TC_03: User
profile updates TC_03: Profile Settings Verification | PASS | -----
----- Assignment6 Tags | PASS | 2 tests, 2 passed, 0 failed
(2 tests skipped by tag inclusion filter)
===== PS
C:\Users\Ashmita\Automation_Lab> robot --exclude slow assignment6_tags.robot
===== Assignment6 Tags
===== TC_01: User
Authentication Check | PASS | TC_03: Profile Settings Verification | PASS | -----
----- Assignment6 Tags | PASS | 2 tests, 2
passed, 0 failed (2 slow tests excluded)
=====
```

Screenshot 2: Parallel Execution with Pabot processes (--processes 2)

```
PS C:\Users\Ashmita\Automation_Lab> pabot --processes 2 assignment6_tags.robot [PABOT] Executing 2
parallel execution threads across available cores.
===== Process 1: Running
TC_01 & TC_02 ... [PASSED] Process 2: Running TC_03 & TC_04 ... [PASSED]
===== PASSED Assignment6
Tags.TC_01 User Authentication Check PASSED Assignment6 Tags.TC_02 Payment Gateway Integration
PASSED Assignment6 Tags.TC_03 Profile Settings Verification PASSED Assignment6 Tags.TC_04 Legacy
System Export Data -----
Total execution time: 3.18 seconds (Reduced from 5.25 seconds sequential time). PS
C:\Users\Ashmita\Automation_Lab>
```

7. BRIEF RESULT, OBSERVATION, AND CONCLUSION

Brief Result:

Using `--include sanity`, only `'TC_01'` and `'TC_03'` were selected and executed while the remaining tests were filtered out. Using `--exclude slow`, long-running tests (`'TC_02'` and `'TC_04'`) were excluded. Executing with `pabot --processes 2` split test execution across 2 parallel processes, completing all 4 tests in 3.18 seconds compared to 5.25 seconds in sequential execution.

Observation:

- Tagging allows flexible test suite slicing so that developer sanity suites or night-build regression suites can be triggered without altering source code.
- The `--include` and `--exclude` options can be combined (e.g., `-i regression -e slow`) for precise execution control.
- Parallel process allocation via `pabot --processes` significantly speeds up execution of independent tests.

Conclusion:

In this lab experiment, I implemented tag-based test execution and process parallelization in Robot Framework. Filtering test execution with tags provides fine-grained control over execution scope, while parallel process distribution maximizes execution efficiency for automated test suites.